

--	--	--	--	--	--	--	--

Question 1: Problem Statement

You are working on a **corporate audit platform** that validates numeric console reports generated by legacy systems.

Since the system cannot compare visual console output directly, every report must be returned as a **collection of text rows**, where:

- Each row is stored as a **string**
- The entire report is returned as an **array/list of strings**
- The layout must remain **vertically and horizontally symmetric**
- Numbers represent **column positions**

Your task is to generate the expected report layout based on a given integer N.

Input Format

- A single integer N representing the scale of the report.

Output Format

- Return an **array/list of strings**
- Each string represents **one row** of the report

Example 1

Input

3

Output

```
[  
  " 1",  
  " 121",  
  "12321",  
  " 121",  
  " 1"  
]
```

Question 2: Problem Statement

Problem Statement

You are given an integer array representing incremental changes applied sequentially to a value.

An initial **positive** value is chosen before applying these changes.

Each element of the array is added one by one to the current value.

Your task is to determine the **minimum positive starting value** such that the running value **never becomes zero or negative** at any point during the process.

Input Format

- An integer array arr of size n

Output Format

- Return a single integer representing the **minimum positive starting value**

Example 1

Input:

arr = [-3, 2, -3, 4, 2]

Output:

5

Explanation:

Starting with a value of 5:

- $5 + (-3) = 2$
- $2 + 2 = 4$
- $4 + (-3) = 1$
- $1 + 4 = 5$

Date: 26/02/2026
Duration: 1 hour 45

CONTINUOUS ASSESSMENT 1
II YEAR – SEMESTER 4
CSE23AE204 - PCP III
(B.Tech. E01,E02,E03,E05,E06)(B.Sc E61)

SET A

Question 3: Problem Statement

Problem Statement

You are developing a **discount calculation module** for an e-commerce platform. The platform processes **one order at a time**, provided as a **space-separated string**. Each order belongs to a specific customer category and follows different discount rules.

The system must:

- Identify the order type from input
- Apply discounts based on customer category
- Calculate the final payable amount
- Use **inheritance and polymorphism** to support multiple order types
- Use **method overloading** to apply different discount strategies
-

Discount Rules

Regular Order

- A percentage-based discount is applied on the original price.
- The discounted amount is subtracted from the original price.
- The remaining value is the final payable amount.

Prime Order

- A percentage-based discount is applied on the original price.
- The discounted amount is subtracted from the original price.
- An additional fixed prime discount is further subtracted.
- The remaining value is the final payable amount.

Input Format

A single **space-separated string**.

Regular Order

REGULAR <CustomerName> <Price> <DiscountPercentage>

Prime Order

PRIME <CustomerName> <Price> <DiscountPercentage>
<PrimeDiscount>

Output Format

Return a **double value** representing the **final payable amount**.

Example 1

Example 1

Input

```
{ "order": "REGULAR Ravi 2000 10" }
```

Output

1800.0

Explanation

A percentage discount is applied on the original price and subtracted to get the final payable amount.